

Preparatory Exercises for the Decorators Chapter

This work must be submitted before Friday 01st May 2026, 20PM to my email address
Prof SD Eyono Obono

Prerequisites

Write each of the following programs, try to understand it, execute it, and analyse its output to learn the decorators concept.

Code to decorate a function with the help of another function, without parameters

```
from functools import wraps

def square_function_decorator(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        result = f(*args, **kwargs)
        return result**2
    return wrapper

@square_function_decorator
def example_function(n):
    return ((2*n) + 1)

print("\nDecorated Function value for variable 1")
print(example_function(1))
```

Code to decorate a function with the help of another class, without parameters

```
from functools import wraps

class SquareClassDecorator(object):
    def __init__(self):
        super().__init__()

    def __call__(self, f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            result = f(*args, **kwargs)
            return result**2
        return wrapper

@SquareClassDecorator()
def example_function(n):
    return ((2*n) + 1)

print("\nFunction value for variable 1")
print(example_function(1))
```

Code to decorate a function with the help of another function, with parameters

```
from functools import wraps

def example_function_powerp_function_modifier(p):
    ''' Raises a function to the power p '''
    def decorate(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            result = f(*args, **kwargs)
            return result**p
        return wrapper
    return decorate

@example_function_powerp_function_modifier(4)
def example_function(n):
    return ((2*n) + 1)

print("\nDecorated Function value for variable 1")
print(example_function(1))
```

Code to decorate a function with the help of another class, with parameters

```
from functools import wraps

class PowerpDecorator(object):
    def __init__(self, p):
        self.p = p

    def __call__(self, f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            result = f(*args, **kwargs)
            return result**(self.p)
        return wrapper

@PowerpDecorator(4)
def example_function(n):
    return ((2*n) + 1)

print("\nFunction value for variable 1")
print(example_function(1))
```

Exercise

- 1) Write a first Python program with a get_DP function to return a DP based on two tests marks with equal weights. Then add another function to your program to decorate the get_DP function for it to return not only the value of the DP, but also a message indicating whether the student with the given DP qualifies for the exam or not. It is assumed that the minimum DP to qualify for the exam is always 40%.
- 2) Write a second Python program with a get_DP function to return a DP based on two tests marks with equal weights. Then add another class to your program to decorate the get_DP function for it to return not only the value of the DP, but also a message indicating whether the student with the given DP qualifies for the exam or not. It is assumed that the minimum DP to qualify for the exam is always 40%.
- 3) Write a third Python program with a get_DP function to return a DP based on two tests marks with equal weights. Then add another function to your program to decorate the get_DP function for it to return not only the value of the DP, but also a message indicating whether the student with the given DP qualifies for the exam or not. It is assumed that the minimum DP to qualify for the exam varies as indicated by the following table.
- 4) Write a fourth Python program with a get_DP function to return a DP based on two tests marks with equal weights. Then add another class to your program to decorate the get_DP function for it to return not only the value of the DP, but also a message indicating whether the student with the given DP qualifies for the exam or not. It is assumed that the minimum DP to qualify for the exam varies as indicated by the following table.

Faculty Type	Typical DP Minimum	Examples of Universities
Humanities / Social Sciences / Education / Theology	40%	UCT Humanities, Stellenbosch Arts & Social Sciences, NMU Humanities, WSU Humanities & Education, UP Humanities, Stellenbosch Theology (40%)
Commerce / Law / Business / Management Sciences	40%; 45%	UCT Commerce (40%), Stellenbosch EMS (40%), Wits Commerce/Law (45%), UFS Commerce/Law (45%), WSU Business & Law (40%)
Science / Natural & Agricultural Sciences	45%; 50%	Wits Science (50%), UP Natural Sciences (50%), Stellenbosch Science (50%), NMU Science (45%), WSU Natural Sciences (45%)
Engineering	50%	UCT Engineering (50%), Stellenbosch Engineering (50%), UP Engineering (50%), NMU Engineering (50%), WSU Engineering (50%)
Health Sciences / Medicine / Nursing	50%	UCT Health Sciences (50%), Stellenbosch Medicine (50%), UP Health Sciences (50%), Wits Health Sciences (50%), NMU Health Sciences (50%), WSU Health Sciences (50%)